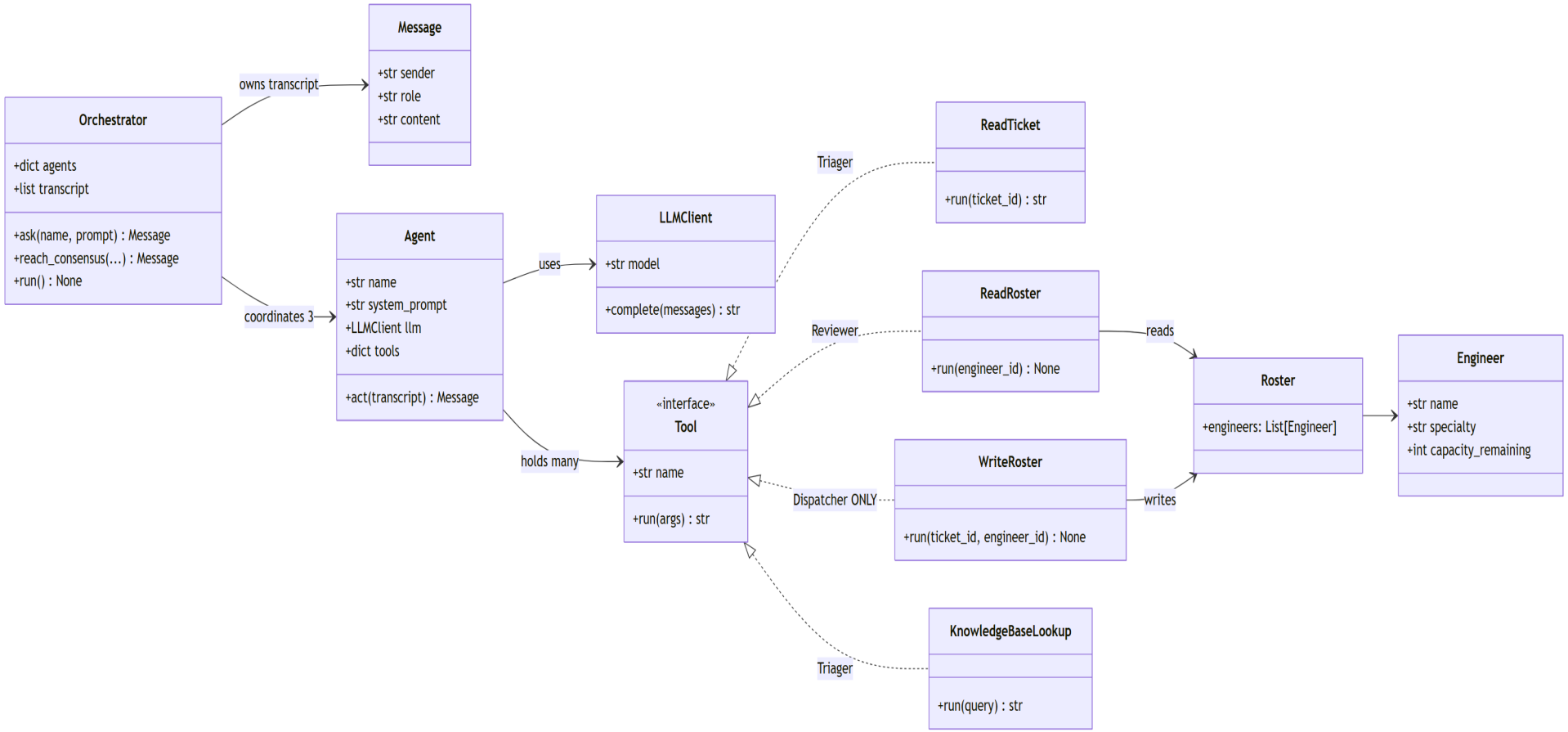
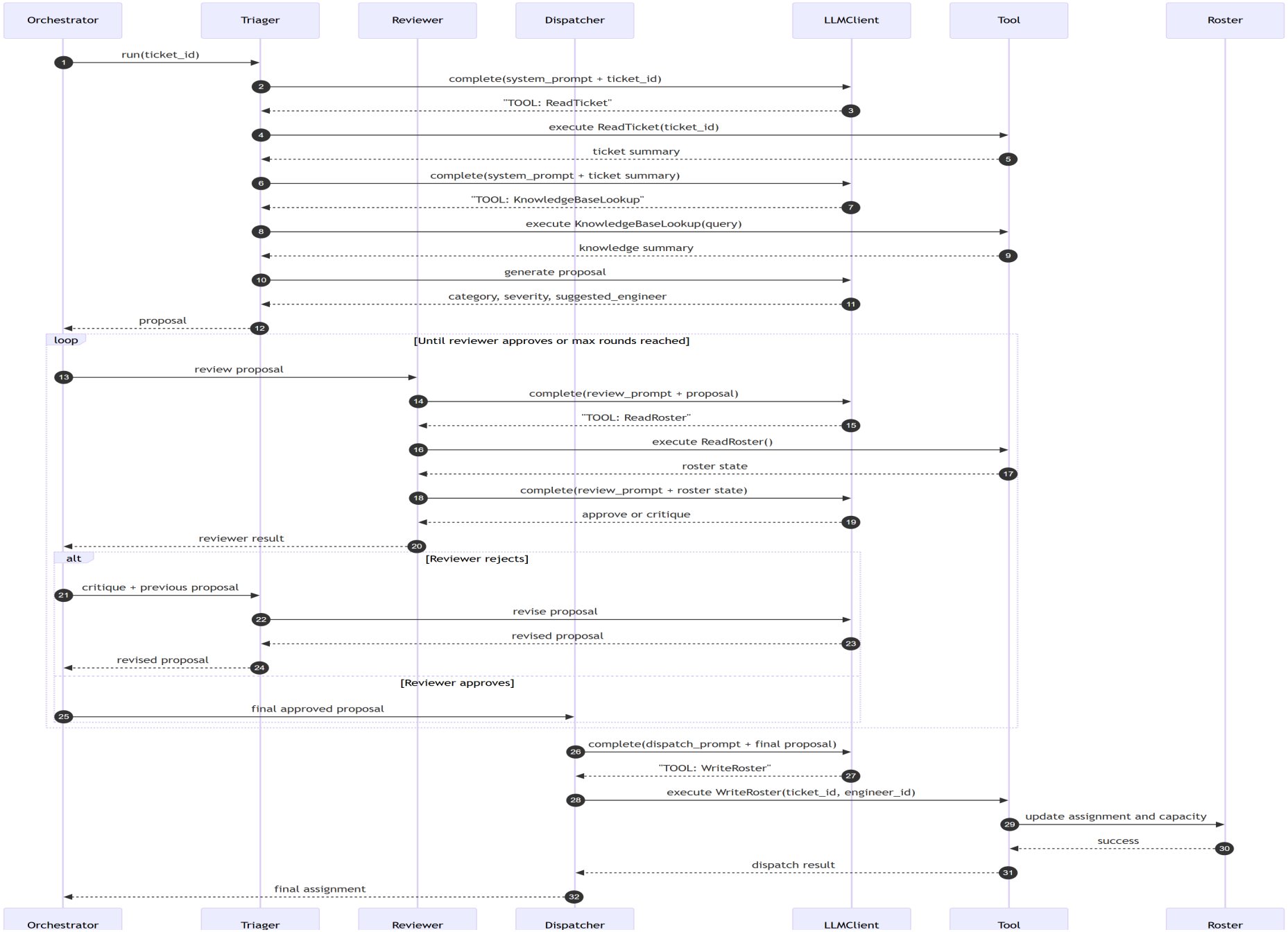
Design Document

Ryan Webster

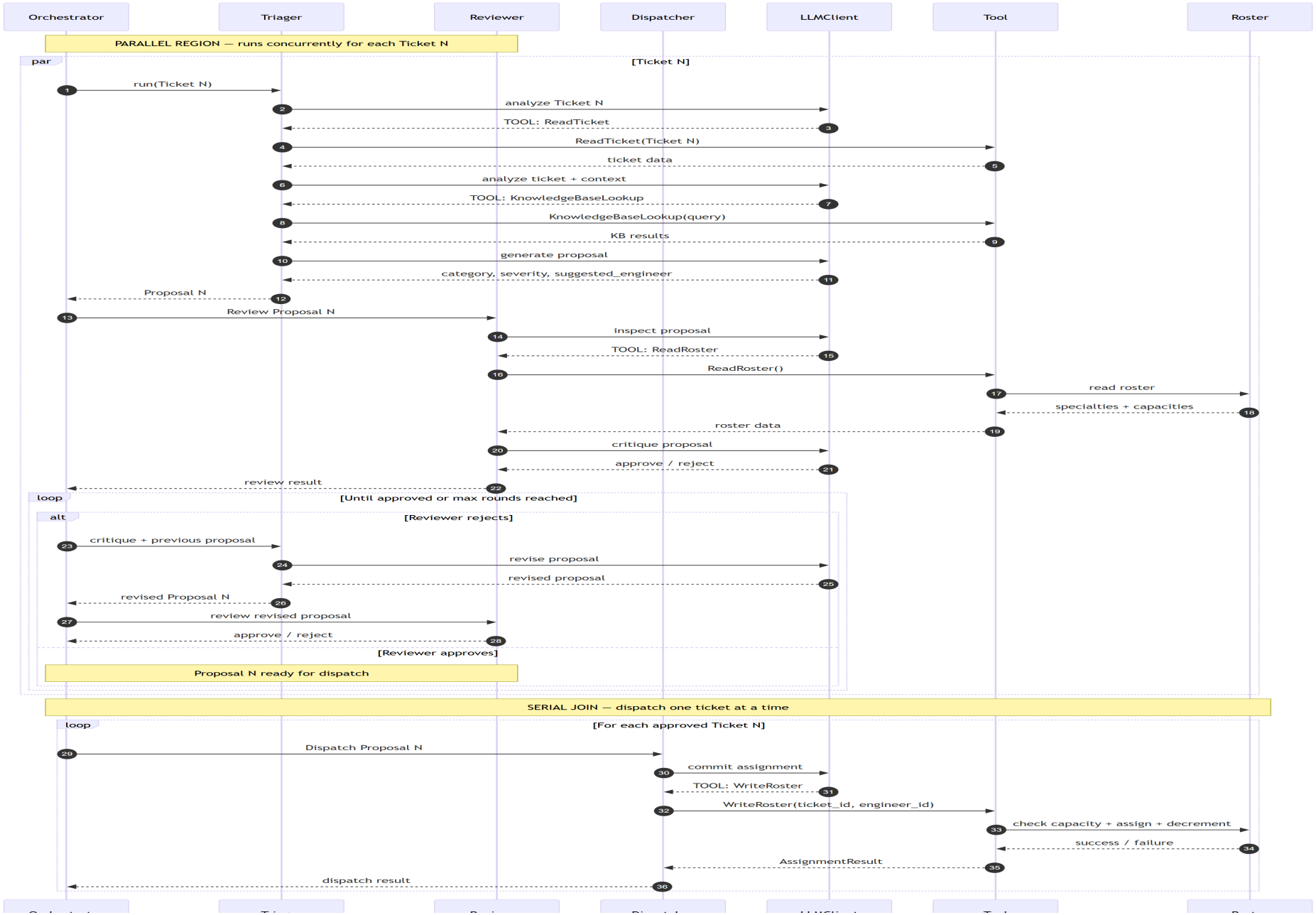
Deliverable A



Deliverable B



Deliverable C



Deliverable D

Question 1

I would allow the Triager to execute proposals in parallel as tickets are independent of each other. Additionally, the Reviewer can then independently check for the proposal's severity, duplicaticity, specialty fit, and availability. These steps can be done in parallel because they are read only operations. For the serial join, I'd reconcile the Triager and Reviewer's outputs and have the Dispatcher update the Roster. This must be done sequentially because of the race condition that exists if capacity remaining is being decremented.

Question 2

The shared mutable state is the Roster, or more specifically, the Engineer's capacity remaining. The arbiter is the Dispatcher, which is the only agent allowed to mutate the roster. Without the arbiter, two tickets could be assigned to a specific engineer with one slot left, essentially making the capacity remaining negative and over assigning that specific engineer. My design prevents this by only approving tickets through the Dispatcher serially.

Question 3

The Traiger's single duty is proposing the ticket's category, severity, and suggested_engineer with tools ReadTicket and KnowledgeBaseLookup. The Reviewer's single duty is to critique the Triager's proposal and verify that the proposed engineer has the right specialty and available capacity with tool ReadRoster. The Dispatcher's single duty is to commit the approved assignment to shared state with the tool WriteRoster.

If you let the Triager also use the Dispatcher's functionality, it could automatically commit its proposal without letting the Reviewer check first.

Question 4

Add an Escalator agent with a prompt stating that "If severity == 1 and no qualified engineer has capacity, page a human responder, otherwise do nothing". Give it the ReadRoster tool and a PageHuman tool. Have the agent take the Reviewer's output and if there is a qualified engineer, go as planned, and if not, page human. I wouldn't modify the current Roster class as it has everything you need.

Final Question

Something that I took away from this lab is that multi-agent design is indeed much more complex than a single agent. What I found impactful about multi-agent systems is the ability to divvy up your work amongst different agents and how it affects verification of work, safer tool calls, scaling, debugging, etc.